Mansi Bishoy

Event Management System: Design & Features

Exploring MVC Architecture and OOP Principles

Problem Statement

Manual event tracking often leads to data inconsistency and a lack of type-safety. This project implements a structured MVC-based system to automate validation and ensure reliable storage of event and workshop data.

Introduction to Event Management

Common Scheduling Pitfalls

- Traditional systems lack **type-safety** and treat all events as generic entries.

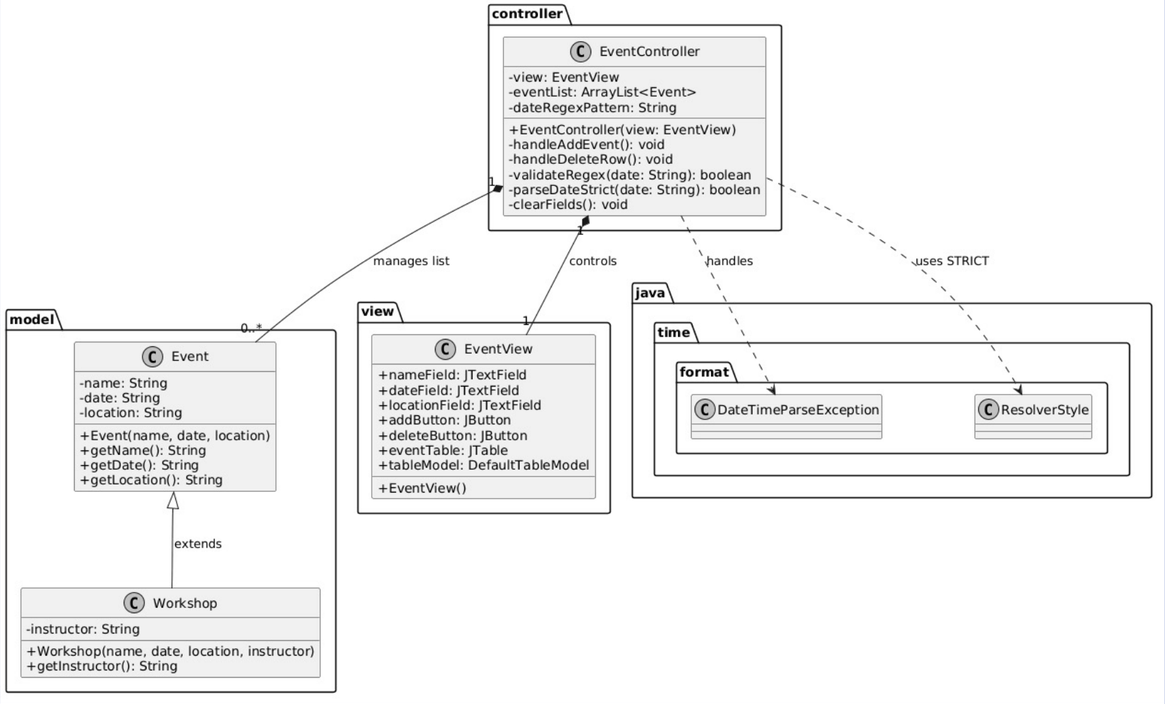
Calendar Awareness Issues

- Logical errors like invalid dates (e.g., February 30th) are often accepted.

Need for Robust Solutions

- A desktop-based system addresses these deficiencies with strict validation and modular design.

System Architecture Overview



MVC Pattern for Separation

Model, View, and Controller layers keep logic, UI, and data distinct.

Modular Package Structure

Code organized into controller, model, and view packages for clarity.

Interaction Flow Enhances Maintainability

User actions trigger controller logic, ensuring robust validation and updates.

Technical Highlights

Design Decision : Implementation of Strict Data Validation using java.time.format.ResolverStyle.STRICT.

Code Logic: * Utilizes Regex (^\\d{2}/\\d{2}/\\d{4}$) for initial format verification.

Employs DateTimeFormatter with the uuuu year pattern to handle proleptic years.

Why Chosen:

Data Integrity: Standard parsers often "leniently" accept invalid dates (like Feb 30th) by rolling them into the next month.

Robustness: Ensures the system only accepts real calendar days, preventing "garbage data" from entering the Model.

Inheritance Streamlines Event Types

Workshop class extends Event, sharing common fields.

Encapsulation Protects Data Integrity

Private fields restrict direct access; getters/setters control changes.

Polymorphism Enables Dynamic Handling

System distinguishes between Event and Workshop at runtime.

Validation & Error Handling

Robust Input Validation

- Employs Regex to enforce DD/MM/YYYY date format
- Utilizes Java Time API with **STRICT** parsing for real dates

Edge Case Handling

- Detects invalid dates like February 30th and month boundaries
- Leap year logic ensures only valid dates are accepted

User Feedback & Error Management

- Immediate warnings for incorrect patterns or invalid selections
- Clear prompts prevent workflow interruptions

Takeaways & Future Scope

Robust Architecture

- MVC pattern ensures clear separation of concerns
- OOP principles like inheritance and encapsulation boost maintainability

Strict Data Validation

- Regex and Java Time API enforce **type-safety** and chronological accuracy

Scalability & Adaptability

- Modular design allows easy upgrades and integration

Future Enhancements

- SQL database integration for persistent storage
- Security layer to restrict sensitive actions